MLX CPU backend: measured speedups on the perf/cpu branch

Fork: github.com/shubhxho/mlx, branch **perf/cpu**. Baseline: upstream ml-explore/mlx commit 96fbf67. Every number in this document was measured. None are projected or estimated.

Summary

Ten commits make the CPU backend multithreaded for elementwise ops, copies, reductions, softmax and sort. They also make the polynomial exp vectorizable with GCC on x86, use multiple accumulators in reductions, and tile transposes. On an 8 vCPU Xeon, changed ops run 2.3x to 63x faster than upstream. Softmax is 21x faster, exp 26x, sums and max about 20x, transposes 20x, sort 7x, and add/mul about 3.5x. Test results match upstream, plus 11 new targeted tests.

Hardware and build

Intel Xeon VM: 8 vCPU (1 thread per core), L2 16 MiB total, L3 320 MiB, 16 GB RAM, Linux, no GPU. GCC 13 (conda-forge) at -O3 with the default x86-64 target, OpenBLAS. CPU-only build (Metal and CUDA off), Release, shared library, Python 3.12 bindings. The baseline and the branch were built identically.

Method

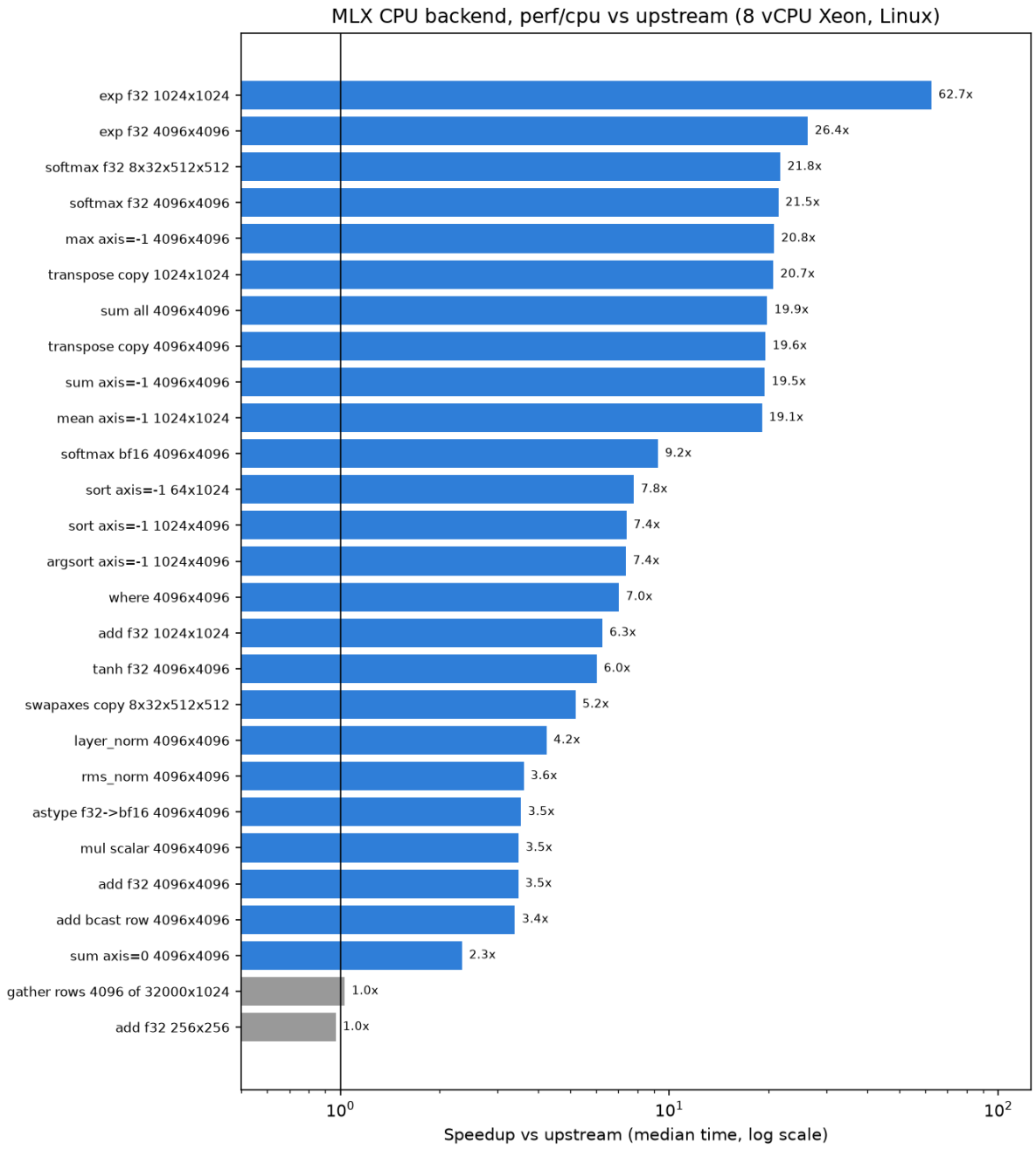
benchmarks/python/cpu_ops_bench.py runs 5 warmup evaluations per op, then 15 rounds of 10 mx.eval calls, and reports the median per-call time with the min..max of the per-round means. Inputs are fixed-seed and reused. The baseline was run twice. 'Before' is the second run, and the first run is shown for scale. Memory-bound single-threaded ops varied by up to ~15% between the two baseline runs. '1 thread' is the branch with MLX_CPU_NUM_THREADS=1, which isolates the single-threaded changes.

Results (median ms per call)

Op	Before (run 1)	Before (run 2)	After	Speedup	After 1 thread	Speedup 1 thread
add f32 256x256	0.021	0.022	0.022	0.97x	0.021	1.02x
add f32 1024x1024	0.415	0.435	0.069	6.27x	0.406	1.07x
add f32 4096x4096	33.668	25.484	7.334	3.47x	25.850	0.99x
add bcast row 4096x4096	26.286	22.567	6.647	3.40x	21.135	1.07x
mul scalar 4096x4096	27.260	23.012	6.600	3.49x	24.941	0.92x
exp f32 1024x1024	14.646	14.439	0.230	62.69x	1.787	8.08x
exp f32 4096x4096	265.770	258.690	9.809	26.37x	46.580	5.55x
tanh f32 4096x4096	100.238	96.783	16.071	6.02x	95.569	1.01x
where 4096x4096	115.256	111.500	15.860	7.03x	93.911	1.19x
astype f32->bf16 4096x4096	20.070	14.408	4.068	3.54x	14.211	1.01x
sum all 4096x4096	9.418	8.742	0.440	19.85x	2.953	2.96x
sum axis=-1 4096x4096	8.656	8.589	0.441	19.49x	2.971	2.89x
sum axis=0 4096x4096	3.209	2.801	1.194	2.35x	3.184	0.88x
max axis=-1 4096x4096	17.143	17.131	0.822	20.85x	6.259	2.74x
mean axis=-1 1024x1024	0.558	0.575	0.030	19.14x	0.142	4.04x
softmax f32 4096x4096	280.837	277.828	12.943	21.46x	69.205	4.01x
softmax f32 8x32x512x512	1100.454	1093.903	50.256	21.77x	311.466	3.51x
softmax bf16 4096x4096	279.786	269.483	29.155	9.24x	195.399	1.38x
rms_norm 4096x4096	61.730	59.788	16.572	3.61x	48.056	1.24x
layer_norm 4096x4096	122.360	109.271	25.787	4.24x	98.873	1.11x
transpose copy 4096x4096	195.316	184.504	9.408	19.61x	44.146	4.18x
transpose copy 1024x1024	4.023	3.935	0.190	20.72x	1.083	3.63x
swapaxes copy 8x32x512x512	183.849	170.868	32.965	5.18x	190.604	0.90x
gather rows 4096 of 32000x1024	1.204	1.282	1.243	1.03x	1.235	1.04x
sort axis=-1 1024x4096	208.089	204.777	27.629	7.41x	208.003	0.98x
argsort axis=-1 1024x4096	253.084	245.353	33.239	7.38x	244.989	1.00x
sort axis=-1 64x1024	2.811	2.808	0.360	7.81x	2.858	0.98x

Every changed op improves by more than the noise. gather (unchanged) and add 256x256 (kept single-threaded on purpose, below the 128 KB-per-thread grain) are about 1.0x. Single-threaded, no op got slower beyond the spread of the two baseline runs. Arrays of 64 MB fit in this machine's 320 MiB L3, so memory-bound gains are higher than on machines with small caches.

Speedup chart



Changes (one commit each)

Commit	Change
1ff67f3	Fork-join worker pool + parallel_for: generation-tagged tickets, nested/concurrent regions run serially, short spin before sleeping, MLX_CPU_NUM_THREADS override.
1e6f08a	Unary ops split across threads; inner loop kept out of line so GCC vectorizes it.
7b742b9	Binary ops (contiguous and broadcast) split across threads.
81f05c1	Ternary ops (where) split across threads.
71fff91	32x32 tiled transpose-like copies; copies and fills split across threads.
c312552	Simd math vectorizable with GCC on x86: multiply-add instead of libm std::fma without FMA, exact branch-free float floor, -fno-trapping-math on GCC (clang's default; results unchanged).
47667fe	8 independent accumulators in contiguous reductions without a SIMD type (also more accurate).
4994380	Reductions split across threads (all-reduce, rows, strided blocks/columns).
eb2a1db	Softmax rows split across threads.
25766a1	Sort/argsort rows in parallel.
fe0fa8e	Targeted tests vs NumPy; benchmark script.

Correctness

Suite	Upstream 96fbf67	perf/cpu
C++ tests (each case separately)	225 / 243 pass	225 / 243 pass (same 18 failures)
Python tests	775 run: 8 fail, 1 error, 60 skip	786 run: same 8 fail + 1 error, 60 skip
New test_cpu_parallel.py	10 / 11 (fails sum accuracy)	11 / 11

The shared failures are the mx.compile tests (all 18 in C++, 8 of 9 in Python) plus test_vmap.test_broadcast_axes_vmap. They fail identically on upstream in this environment. exp's max relative error over [-87, 87] is 3.85e-6 on both builds. The 16M-element float32 sum error vs float64 drops from 1.1e-4 to 8.5e-7. Outputs with 1 and 4 threads are bit-identical for elementwise ops, copies and sorts. Reassociated float reductions can differ from upstream in the last bits.

Limits

• CPU only, Linux x86 only. No Metal, CUDA or Apple Silicon numbers. The Apple Silicon behaviour of the thread pool is untested.

• The fma/floor/-fno-trapping-math change only affects GCC x86 builds without FMA/SSE4.1 enabled at compile time.

• mx.compile fused CPU kernels don't work in this environment on upstream either, so compiled graphs weren't benchmarked.

• One machine, an 8 vCPU VM with a very large L3. The parallel grain is a fixed constant, not tuned per op.

• Intermediate commits weren't built one at a time; the full branch was built and tested.

• Not touched: BLAS/matmul, conv, gather/scatter, scans, quantized ops.

Reproduce

Build with cmake -DMLX_BUILD_METAL=OFF -DMLX_BUILD_CUDA=OFF -DMLX_BUILD_PYTHON_BINDINGS=ON. Then run python benchmarks/python/cpu_ops_bench.py --json after.json, do the same on upstream to get before.json, and run python benchmarks/python/cpu_ops_report.py before.json after.json --chart speedup.png. Full commands are in PERF.md.